

Algorithm Interview Guide + Rubric

Background

This is a very new scoring system, and I hope that you use it more as a heuristic than a strict regulation, especially as we calibrate this (please help us gather data on the calibration here)

I intend for this scoring system to differentiate levels based on the quantity and quality of proficiencies demonstrated by the candidate, though it bears stating that we are actively reviewing and analyzing this scoring system, so please consider this to be a work in progress as you review candidates**.** For now, we are counting and calibrating to determine if this rubric has value in interviewing candidates.

We are using several principles to guide evaluation of candidates, and for each principle:

- *Red Flags* indicate fundamental weaknesses and should be minimal or absent, especially for higher levels.
- *Yellow Flags* show areas where the candidate has some understanding but lacks proficiency, which should decrease as experience and level increase.
- *Baseline* represents the minimum expected proficiency. Meeting or exceeding this in more areas is required as one moves up levels.
- *Bonus* serves as a strong indicator of a candidate's proficiency and excellence in certain areas. The scoring mechanism rewards exceeding baseline expectations, which should become more common at higher levels.

Also note that these categories are mostly mutually exclusive, with the exception being Bonus can only be awarded if the Baseline is met. In other words, if a candidate gets a yellow or red flag, they did not meet the Baseline.

How to conduct the interview

Conducting the interview

1. Briefly introduce yourself (e.g., how long have you been at Datavant, what team/org, what your team does) and ask the candidate for a brief intro
2. Introduce the interview

    a. “This will be a coding interview where we’ll work through a few coding problems one by one. More performant solutions are preferable, but your primary focus should be on getting working solutions as quickly as possible. Additionally, try to write clean and well-organized code, and vocalize your thought process when possible. Does that all sound good?” -> NOTE we do not tell the candidate how many problems there are up front
3. Click `Create Pad` in the Coderpad pads tab and switch the language to whatever the candidate says they want to do the interview in (Pythhttps://app.coderpad.io/dashboard/padson, Java, JS)
4. Click on the relevant Algos question in the question bank -> Starter Code -> Copy the first question
5. Copy the next question when they finish the previous one, and repeat
6. NOTE for question 3 specifically, if the candidate’s implementation just uses built-ins like pop/insert or their equivalents the interviewer should ask if they can come up with an implementation without those methods. This is to get a better signal on the candidate’s ability to implement things of greater complexity (and their understanding of what built-ins might look like under the hood)

Finishing the interview + Scorecard

1. **Remember to click End Interview in the Coderpad**, which makes it so that the candidate can no longer see the questions (helping prevent leaks) and allows the interviewer to get a playback if they want to take another look at how long the candidate took for various steps or paths they went down
2. In the Overall Recommendation section of the scorecard, copy and fill out the Overall and Breakdown sections. Then use the guidance in Expectations for Levels to assist in making an overall decision, which you can include in a tl;dr/summary at the top of the Overall section.

Expectations for Levels

Entry or Mid-Level:

Yes:

- Up to 1 yellow flag is acceptable, and 0 red flags (i.e., 3-4 Baseline points)

Strong Yes:

- All Baseline points are met (0 yellow or red flags) and the Bonus for **`Ability to solve problems`** was met

Senior or Staff:

Yes:

- All Baseline points are met (0 yellow or red flags)

Strong Yes:

- All Baseline points are met (0 yellow or red flags), the Bonus for `Ability to solve problems` was met, and one other Bonus was met

Overall

Summary: <Write an overview of the specific strengths/weaknesses a candidate showed during the interview. If there are signals/important observations that are not expressed in the template, add them here. You can also mention how the candidate fared with respect to the guidelines mentioned in the Expectations for Levels section to justify your decision>.

Baseline: out of 4
Bonus: out of 4
Yellow flags: out of 4 [mutually exclusive with Baseline]
Red flags: out of 4 [mutually exclusive with Baseline]

Breakdown

Ability to solve problems

- **[ ] Baseline**: Is able to complete the first 3 questions with no hints (or one very minor one). For question 3, they are also able to come up with a solution that does not make use of built-ins like pop/insert
- **[ ] Bonus**: Is able to complete the 4th question
- **[ ] Yellow Flag**: Is only able to complete the first 3 questions with some significant hinting
- **[ ] Red Flag:** Is unable to complete the first 3 questions even with hinting

Ability to write clean and idiomatic code

- **[ ] Baseline**: Implementations are reasonably clean/readable/maintainable, and candidate demonstrates a reasonable understanding of common syntax/idiomatic expressions in their language of choice
- **[ ] Bonus**: Demonstrates strong proficiency in writing clean code and a strong understanding of idiomatic expressions, to the point where it is clear the candidate has a lot of production-quality coding experience
- **[ ] Yellow Flag**: Occasionally writes code that is unclean/hard to read or that contains syntax errors and/or unidiomatic expressions AND does not fix any of this themselves without prodding
- **[ ] Red Flag:** Consistently struggles with syntax, and/or consistently writes unclean/unreadable/unidiomatic code, suggesting not a lot of production-quality coding experience

Understanding of algorithmic complexity

- **[ ] Baseline**: If implementations are very suboptimal, is able to recognize that this is the case and describe better implementations (with perhaps some minor prodding)
- **[ ] Bonus**: Implements optimal or near-optimal solutions without prodding and/or is able to explain tradeoffs (i.e., an optimal solution may have more code complexity/maintainability issues that may not be justified if the code is only being used at small scales, or in a real world setting this part of the code is unlikely to be a bottleneck)
- **[ ] Yellow Flag**: If implementations are very suboptimal, is able to recognize that this is the case but can't describe potentially better approaches
- **[ ] Red Flag:** If implementations are very suboptimal, does not recognize that this is the case

Technical Communication Skills

- **[ ] Baseline**: Is able to vocalize their thought process and respond to feedback reasonably well
- **[ ] Bonus**: Demonstrates excellent communication skills, articulating their thoughts/ideas very clearly
- **[ ] Yellow Flag**: Communication often lacks organization or clarity, leading to misunderstandings or the need for repeated explanations.
- **[ ] Red Flag:** Struggles significantly to communicate ideas, understand questions, and/or respond to feedback, leading to frequent miscommunications and inefficiencies.

Appendix

Some additional information on the questions being asked can be found in the old GitHub wiki page for this interview